

前文讲解了随机法、权重随机法、哈希法、一致性哈希法，本文继续讲解轮询法、权重轮询法。

6、轮询法

轮询法的含义：依次遍历每个节点。含义非常简单，实现也很简单。

代码抽象：把全部节点放入节点列表，循环遍历，保持顺序不变。

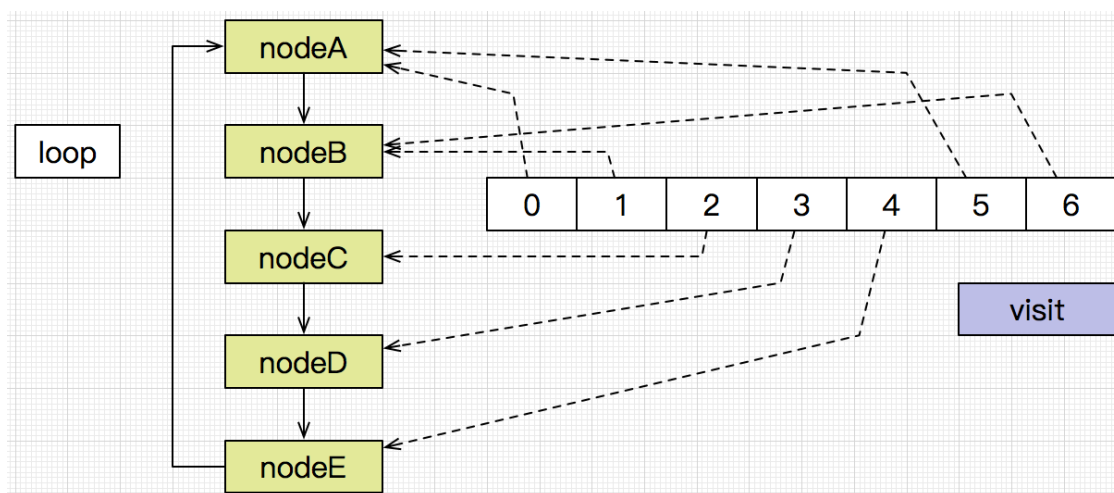
轮询法的英文为 round robin，意为一圈一圈的循环。

轮询法的特点：

使用固定的顺序遍历节点，分散均匀。

易于实现，使用广泛，效果明显。

轮询法的示意图：



轮询法的适用场景：

- 网关。nginx 把流量转发到下游 web 服务器。
- RPC。服务调用方把请求转发到服务提供方。

轮询法的代码实现：

- 节点类

```
@Getter
@Setter
@Accessors(chain = true)
public class Node {
    public String server;// 节点信息
}
```

- 轮询法函数。使用 AtomicLong+nodeList，即可快速实现。

```
// 轮询法
public static Node balanceByRobin(List<Node> nodeList, AtomicLong visit) {
    // 计数加 1
    long seq = visit.getAndIncrement();
    // 获得节点下标。
    int count = nodeList.size();
```

```

    int index = (int) (seq % count);
    // 找到对应节点
    Node node = nodeList.get(index);
    return node;
}

```

- 准备一批节点

```

// 构建一批节点
Node nodeA = new Node().setServer("192.168.0.100");
Node nodeB = new Node().setServer("192.168.0.101");
Node nodeC = new Node().setServer("192.168.0.102");
Node nodeD = new Node().setServer("192.168.0.103");
Node nodeE = new Node().setServer("192.168.0.104");
List<Node> nodeList = Arrays.asList(nodeA, nodeB, nodeC, nodeD, nodeE);

```

- 循环调用与统计

```

public static void loopByRobin(List<Node> nodeList, int maxLoop) {
    // 计数
    Map<Node, AtomicInteger> countMap = new LinkedHashMap<>();
    for (Node node : nodeList) {
        countMap.put(node, new AtomicInteger(0));
    }
    AtomicLong visit = new AtomicLong(0);
    for (int num = 0; num < maxLoop; num++) {
        Node node = balanceByRobin(nodeList, visit);
        // 计数
        AtomicInteger count = countMap.get(node);
        count.incrementAndGet();
    }
    System.out.println("\n loopCount = " + maxLoop);
    for (Map.Entry<Node, AtomicInteger> entry : countMap.entrySet()) {
        System.out.println(entry.getKey().server + " = " + entry.getValue().get());
    }
}

```

- 循环 1 万次，看每个节点的流量

```

// 循环 1 万次
loopByRobin(nodeList, 10000);

```

```

loopCount = 10000
192.168.0.100 = 2000
192.168.0.101 = 2000
192.168.0.102 = 2000
192.168.0.103 = 2000
192.168.0.104 = 2000

```

- 循环 20 万次，看每个节点的流量

```
// 循环 20 万次
```

```
loopByRobin(nodeList, 200000);
```

```
loopCount = 200000
```

```
192.168.0.100 = 40000
```

```
192.168.0.101 = 40000
```

```
192.168.0.102 = 40000
```

```
192.168.0.103 = 40000
```

```
192.168.0.104 = 40000
```

- 从测试结果看出，轮询法使得流量均衡。

循环 1 万次，每个节点的流量为 2000 。

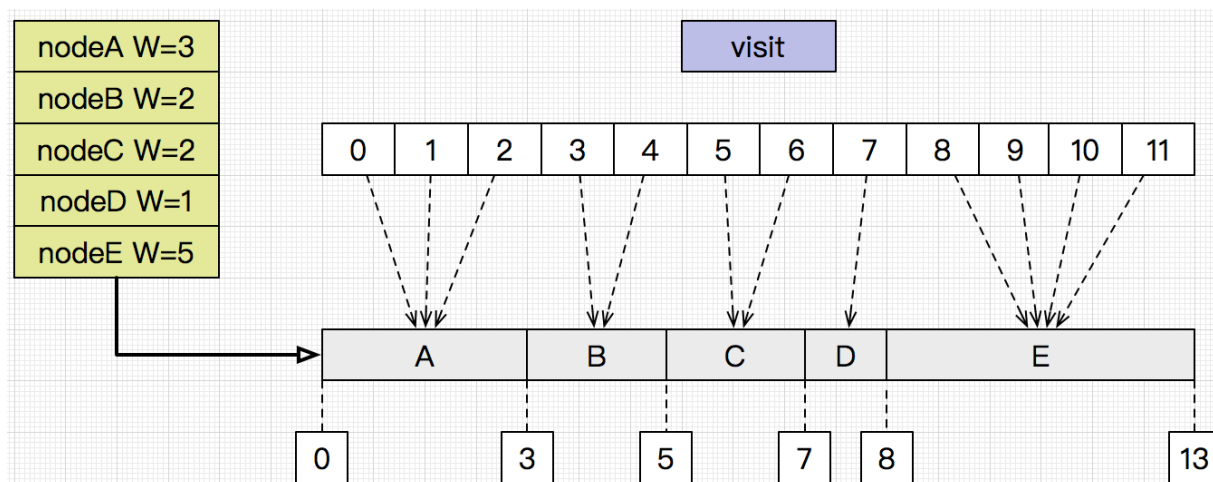
循环 20 万次，每个节点的流量约为 4 万 。

7、权重轮询法

权重轮询法 = 轮询法 + 权重

权重轮询法与权重随机法有相似之处：把每个节点的权重累加起来获得总权重，用总权重计算出权重阈值，用权重阈值找到节点。

权重轮询法的示意图：



权重轮询法的代码实现：

- 节点类

```
@Getter
```

```
@Setter
```

```
@Accessors(chain = true)
```

```
public class NodeWeight {
```

```
    public String server;// 节点信息
```

```
    public int weight;// 权重。从 0 到 100
```

```
}
```

- 权重轮询法函数。使用 AtomicLong+TreeMap，非常高效，有技巧。

```
// 权重轮询法
public static NodeWeight balanceByRobin(List<NodeWeight> nodeList, AtomicLong visit) {
    // 计数加 1
    long seq = visit.getAndIncrement();
    // 使用权重范围，组成 1 个环。
    TreeMap<Long, NodeWeight> weightMap = new TreeMap<>();
    // 累加总权重。同时，给每个 node 分配权重范围。
    int sumWeight = 0;
    for (NodeWeight node : nodeList) {
        long weightFrom = sumWeight;
        sumWeight += node.weight;
        weightMap.put(weightFrom, node);
    }
    // 获得节点下标。
    long index = seq % sumWeight;
    Map.Entry<Long, NodeWeight> entry = weightMap.floorEntry(index);
    // 找到对应节点
    NodeWeight node = entry.getValue();
    return node;
}
```

- 准备一批节点

```
// 构建一批节点
NodeWeight nodeA = new NodeWeight().setServer("192.168.0.105").setWeight(1);
NodeWeight nodeB = new NodeWeight().setServer("192.168.0.106").setWeight(2);
NodeWeight nodeC = new NodeWeight().setServer("192.168.0.107").setWeight(3);
NodeWeight nodeD = new NodeWeight().setServer("192.168.0.108").setWeight(4);
NodeWeight nodeE = new NodeWeight().setServer("192.168.0.109").setWeight(0);
List<NodeWeight> nodeList = Arrays.asList(nodeA, nodeB, nodeC, nodeD, nodeE);
```

- 循环调用与统计

```
// 循环多次。
public static void loopRobin(List<NodeWeight> nodeList, int maxLoop) {
    // 计数
    Map<NodeWeight, AtomicInteger> countMap = new LinkedHashMap<>();
    for (NodeWeight node : nodeList) {
        countMap.put(node, new AtomicInteger(0));
    }
    AtomicLong visit = new AtomicLong(0);
    for (int m = 0; m < maxLoop; m++) {
        // 随机法
        NodeWeight chooseNode = balanceByRobin(nodeList, visit);
        // 计数
        AtomicInteger count = countMap.get(chooseNode);
        count.incrementAndGet();
    }
}
```

```

System.out.println("\n loopCount = " + maxLoop);
for (Map.Entry<NodeWeight, AtomicInteger> entry : countMap.entrySet()) {
    System.out.println(entry.getKey().server + " = " + entry.getValue().get());
}
}

```

- 循环 1 万次，看每个节点的流量

```

// 循环 1 万次
loopRobin(nodeList, 10000);

```

```

loopCount = 10000
192.168.0.105 = 1000
192.168.0.106 = 2000
192.168.0.107 = 3000
192.168.0.108 = 4000
192.168.0.109 = 0

```

- 循环 20 万次，看每个节点的流量

```

// 循环 20 万次
loopRobin(nodeList, 200000);

```

```

loopCount = 200000
192.168.0.105 = 20000
192.168.0.106 = 40000
192.168.0.107 = 60000
192.168.0.108 = 80000
192.168.0.109 = 0

```

- 从测试结果看出，权重轮询法使得流量与权重比率匹配。

权重比率为 1:2:3:4:0。

循环 1 万次，每个节点的流量为 1000/2000/3000/4000/0，等比例换算为 1/2/3/4/0。

循环 20 万次，每个节点的流量约为 2 万/4 万/6 万/8 万/0，等比例换算为 1/2/3/4/0。

权重功能，通常使用列表 ArrayList、红黑树 TreeMap 来实现。

- ArrayList 的优点为更直观更简单，缺点为计算权重阈值需要依次遍历列表，时间复杂度为 $O(N)$ 。
- TreeMap 的优点为平衡二叉树查询高效，时间复杂度为 $O(\log N)$ ，缺点为必须理解红黑树的排序特性，技巧性更高。

这里使用 TreeMap：

- 每个 NodeWeight 对应一个权重范围。
- 保存权重范围使用 `weightMap.put(weightFrom, node)`，从当前 `weightFrom` 开始，到下一个 `weightFrom` 结束，左闭右开。
- 环形功能使用取余法 `long index = seq % sumWeight`，其中 `sumWeight` 表示权重累加。
- 查询权重阈值使用 `weightMap.floorEntry(index)`。